

Lecture 9: TCP/IP & OSI Model

Port numbers deliver data to the correct application, TCP and UDP take different approaches to reliability, and the OSI Model maps onto the real-world TCP/IP Model.

4. Transport Layer

1) End-to-End Delivery / Port-to-Port Delivery



Data reaches the correct device via its IP address — but a user might have many tabs open at once (Wikipedia, YouTube, Pinterest, a live class). Data arrives at the device via IP, but which tab should it go to?

This is exactly what the **Port Number** solves — every website/service uses a different port number, so the same device can distinguish between many simultaneous connections.

[Data | Port No.]

Port Number: Size and Range

Port Number → 16 bits

$2^{16} = 65536$ possible port numbers → range: 0 - 65535

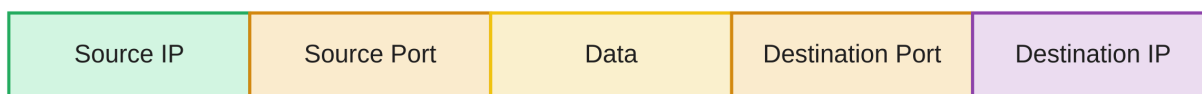
Well Known Ports Example:

http → 80
https → 443
dns → 53

Port Number Categories

RANGE	CATEGORY	DESCRIPTION
0 – 1023	Well Known Ports	Reserved for specific, standard services and protocols
1024 – 49151	Registered Ports	Assigned by IANA (Internet Assigned Numbers Authority) for specific, non-standard purposes
49152 – 65535	Dynamic / Private Ports	Typically used for client-side applications

Packet Structure: IP and Port Together



Example: Opening a Website (HTTPS)

Request:

```
[ 10 | 50000 | data | 443 | 20 ]
```

Reply:

```
[ 20 | 443 | data | 50000 | 10 ]
```

 → here, port 50000 was the tab that was originally open

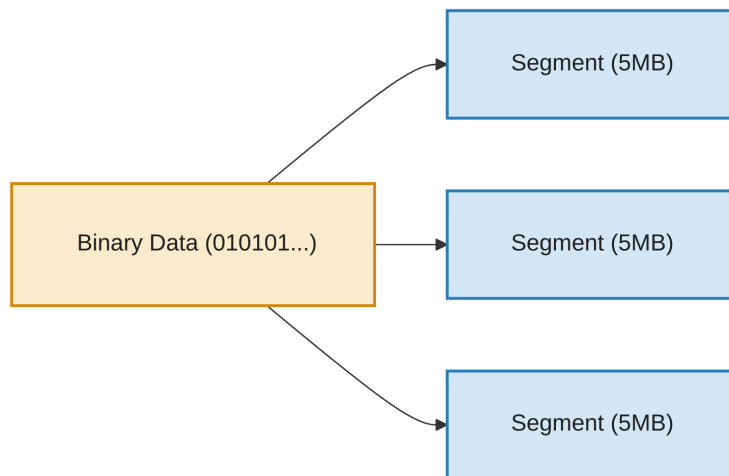
Analogy: A Home With Multiple Members

Home Address (like an IP) → shared by everyone in the house
Each family member (like an application/tab) → needs their own individual identification (like a Port Number)

*The system provides port numbers **dynamically**, and these change with each new connection — this is how data gets routed to the correct tab or application.*

2) Segmentation

The Transport Layer breaks data into **Segments**.



Full Journey of Data Through the Layers

Transport Layer: [60K (port) | data | 443 (port)] → Segment

↓
Network Layer: [IP | 60K | data | 443 | IP]
→ Packet

↓
Data Link Layer: [100 (S.MAC) | 10 (S.IP) | 60K | data | 443 | 20 (D.IP) | 300 (D.MAC)] → Frame

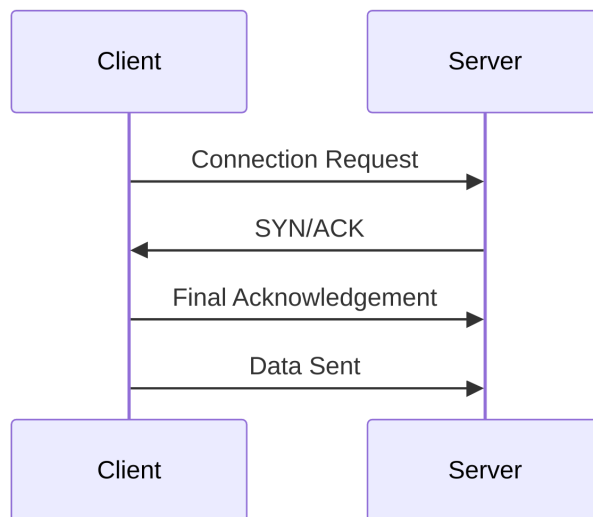
↓
Physical Layer: Actual transfer of data (as raw signals)

- ♦ The **Network Layer** adds the IP Address.
- ♦ The unit's name changes at each layer: Segment → Packet → Frame.

Transport Layer Protocols: TCP & UDP

TCP → Transmission Control Protocol
UDP → User Datagram Protocol

TCP — Connection-Oriented

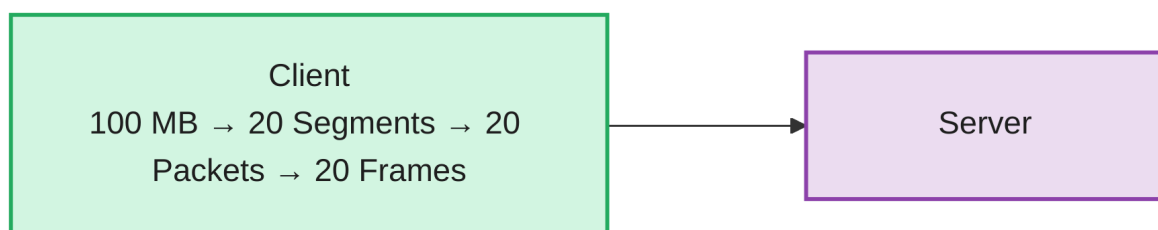


- ✦ This 3-step handshake is called the **3-Way Handshake**, and it forms a connection before any data is sent.
- ✦ Once the handshake completes, actual data transfer begins.

Why Use TCP?

- ✦ Reliable data transfer.
- ✦ Packets are divided and tracked.
- ✦ If a packet is not received, the server informs the client, and the client can resend the missing data.

Flow Control



Example:

18 frames sent successfully, 2 missing.
Server sends acknowledgment: "send 2 more"
Client resends the 2 missing frames → effective/complete connection achieved.

Flow Control happens on an end-to-end basis (between client and server directly) — it does not involve the router.

How does the server know how many packets to expect? This is discussed during the **3-Way Handshake** itself — segment lengths are agreed upon upfront.

Error Checking

Error checking happens directly between the client and server — for example, if 2 of the 20 expected packets are missing, the client is asked to resend them.

UDP — Connectionless



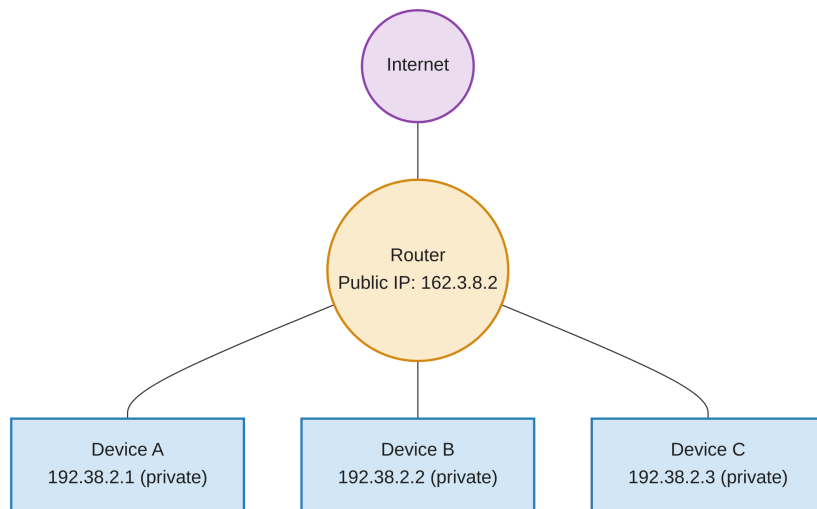
- ✦ No flow control.
- ✦ No way to check if packets are missing.
- ✦ No connection was made beforehand (unlike TCP's handshake).
- ✦ This makes UDP **unreliable** but significantly **faster**, since there's no overhead from handshakes or acknowledgments.

Which protocol is "best" depends entirely on the use case.

PROTOCOL	BEST SUITED FOR	WHY
TCP	Wikipedia, Instagram, Email	All data must be received completely and accurately
UDP	Livestreaming, Online Gaming	Audio/video quality may drop slightly, but data is received without added delay

NAT and Port Numbers — Why Ports, Not MAC Addresses?

NAT (Network Address Translation) lets a router convert private IPs to a shared public IP.



Scenario: Both Device A and Device B want to connect to the same server, using the same destination port, at the same time.

```
Device A request: [ 192.38.2.1 | 50K (port) | data | 80 | 30 ]
Device B request: [ 192.38.2.2 | 50K (port) | data | 80 | 30 ]
```

The router changes the private IP to the public IP for both:

```
[ 162.3.8.2 | 50K | data | 80 | 30 ] → for Device A
[ 162.3.8.2 | 50K | data | 80 | 30 ] → for Device B (same public IP AND same source port!)
```

Problem: Now both look identical to the server — how will the router tell them apart when the reply comes back?

Solution: NAT Also Reassigns Port Numbers

```
Device A's original port (50K) → Router reassigns as 62K
Device B's original port (50K) → Router reassigns as 57K
```

```
Outgoing (Device A): [ Public IP | 62K | data | 80 | 30 ]
Outgoing (Device B): [ Public IP | 57K | data | 80 | 30 ]
```

NAT stores this mapping (which internal device corresponds to which reassigned port number), and the router uses this table to correctly route the server's reply back to A or B specifically.

Why Not Just Use MAC Address Instead of Port Number?

MAC addresses live at the **Data Link Layer** and change at every single hop as a packet moves across the network — they cannot be used for this kind of end-to-end tracking, since they're not stable across the whole journey.

```
[ 1 | SIP | Data | DIP | 2 ] → MAC changes between hop 1 and hop 2  
[ 2 | SIP | Data | DIP | 3 ] → MAC changes again between hop 2 and hop 3
```

This is why **Port Numbers** (Transport Layer) are used instead — they stay consistent for the life of the connection.

5. Session Layer

The Session Layer has 4 responsibilities:

1. **Create Session** — establishing a session for communication.
2. **Authentication** — checking ID and password (login).
3. **Authorization** — determining what a user is allowed to access *after* logging in (e.g., a user can log into their bank account, but cannot arbitrarily change their account balance).
4. **Check-Point** — tracking the progress of an ongoing data transfer.

Check-Point Example

Downloading a 10 GB file → 8 GB completes → connection drops due to a network issue → 2 GB remains

Solution: Resume from the 8 GB checkpoint — no need to re-download the entire 10 GB from scratch.

*This resume-from-checkpoint capability is handled by the **Session Layer**.*

Session Hijack

Example: A YouTube account gets hacked by stealing stored cookies → leading to Session ID theft

6. Presentation Layer

The Presentation Layer has 3 responsibilities:

1. **Encrypt / Decrypt** — e.g. transforming ABC into CDE conceptually (data scrambled for transit).
2. **Data Compression** — e.g. reducing a 10 MB file down to 6 MB before transfer, using algorithms designed for this purpose.

- 3. Data Translation** — converting between formats/encodings the two systems understand, e.g. converting ASCII code → UTF-16.

7. Application Layer

The Application Layer has 3 responsibilities:

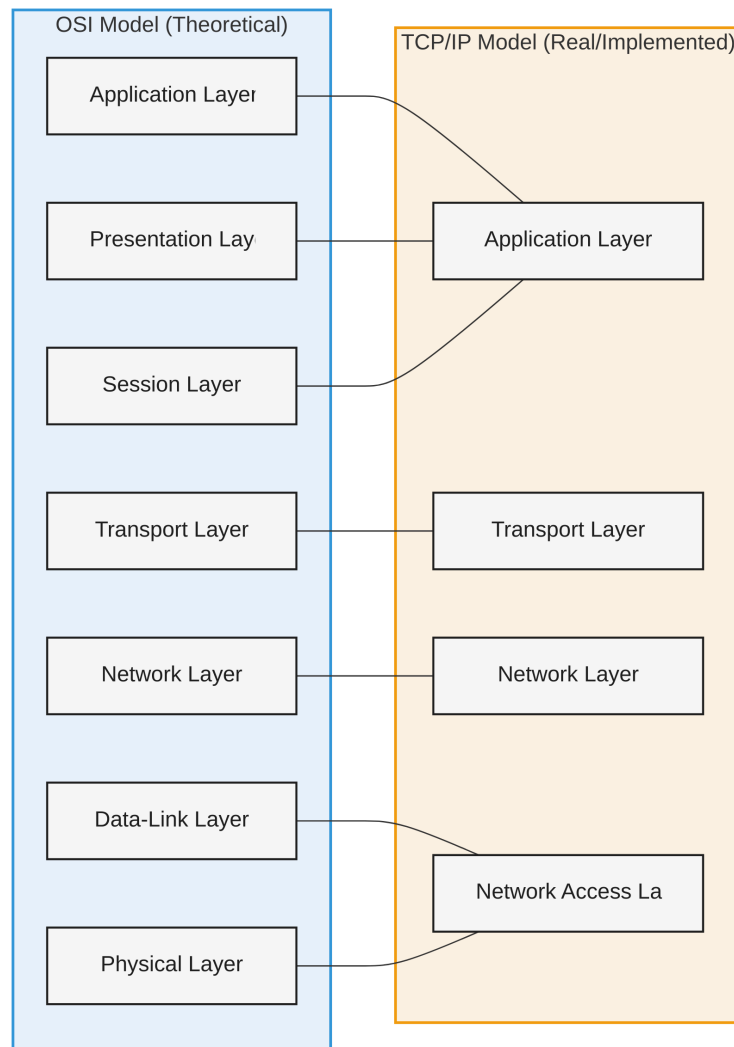
- 1. Provide Network Interface** — this is the layer users actually interact with (e.g., opening a website, seeing the YouTube UI). It ensures the interaction between the user and the website or service.
- 2. Protocols** — includes services like:

PROTOCOL	FULL FORM	PURPOSE
HTTP	HyperText Transfer Protocol	Website/browsing (not secure)
FTP	File Transfer Protocol	File transfer (not commonly used today)
SMTP	Simple Mail Transfer Protocol	Mail transfer (e.g. Gmail)
HTTPS	—	Same as HTTP, but with added security

- 3. Network Transparency** — ensures the user only focuses on using the service, without needing to know or worry about the internal working of how data actually travels.

OSI Model vs TCP/IP Model

OSI is a theoretical model (7 layers) — it describes the concepts in full detail. **TCP/IP** is the practical, real-world implementation — a simplified 4-layer model.



Quick Recap: What Each OSI Layer Does

Application	→ data received / user interaction
Presentation	→ encrypt/decrypt, data translation
Session	→ authenticate / authorize
Transport	→ segments, port numbers
Network	→ IP address
Data-Link	→ MAC address
Physical	→ actual data transfer (raw bits)

The exact same process happens at the receiver's end, but in reverse order — data climbs back up from Physical to Application.

TCP vs UDP — Full Comparison

FEATURE	TCP	UDP
Connection	Connection-Oriented (3-Way Handshake)	Connectionless (no handshake)
Reliability	Reliable	Unreliable
Ordering	Ensures packet order	No ordering of packets
Error Checking	Error checking with retransmission	Basic error checking (no correction)
Flow & Congestion Control	Yes (adjusts based on network conditions)	No flow or congestion control
Speed	Slower, due to overhead	Faster, due to less overhead
Common Use	Web browsing, email, file transfer	Streaming, gaming, DNS, VoIP
Data Segmentation	Breaks data into segments	Sends data as independent datagrams
Data Size	Supports larger data packets	Typically used for small, quick data packets

Summary

- ♦ The Transport Layer achieves port-to-port delivery, using 16-bit port numbers (0–65535) so that data reaching a device via its IP address is routed to the correct application/tab — with port ranges split into Well Known (0–1023), Registered (1024–49151), and Dynamic/Private (49152–65535) categories.
- ♦ Data passes through the layers as: Segment (Transport, has port numbers) → Packet (Network, adds IP address) → Frame (Data Link, adds MAC address) → raw transmission (Physical).
- ♦ TCP is connection-oriented, establishing a session via a 3-way handshake before sending data, and offers reliable transfer with flow control and error checking with retransmission; UDP is connectionless, faster, but unreliable with no flow control — making TCP better for services like web browsing and email, and UDP better for live streaming and gaming.
- ♦ NAT uses port numbers, not MAC addresses, to track which internal device a connection belongs to, since MAC addresses change at every hop and can't be relied on for end-to-end tracking, while port numbers stay consistent for the life of a transport-layer connection.
- ♦ The Session Layer creates sessions, handles authentication (login) and authorization (what a user can access), and manages checkpoints so an interrupted transfer (like a large file download) can resume instead of restarting.
- ♦ The Presentation Layer handles encryption/decryption, data compression, and data translation (e.g. ASCII to UTF-16) between systems.
- ♦ The Application Layer provides the network interface users directly interact with, supports protocols like HTTP, FTP, SMTP, and HTTPS, and ensures network transparency so users don't need to understand the internal workings of data transfer.
- ♦ OSI is a theoretical 7-layer model, while TCP/IP is its practical 4-layer implementation: TCP/IP's Application Layer merges OSI's Application, Presentation, and Session layers, and its Network Access Layer merges OSI's Data-Link and Physical layers — with Transport and Network layers mapping directly between the two models.